

Vault MCP Transit Security Extension

Bob-a-thon Implementation Plan

Extending Vault MCP with Vault Transit support so Bob can guide and execute encryption-as-a-service workflows through Vault.

Prepared for IBM Austin Bob-a-thon 2026

Draft v1

Executive Summary

This project extends the capabilities of Vault MCP by adding support for Vault Transit Secrets Engine operations. With this extension, Bob can act as an agentic security interface for real Vault-backed encryption workflows, including key creation, encryption, decryption, key rotation, ciphertext rewrapping, and data integrity verification.

Instead of only explaining how encryption should work, Bob can now help users perform secure encryption-as-a-service workflows through Vault MCP. This gives developers and security teams a practical way to use Vault Transit without directly managing cryptographic keys or manually constructing Vault API requests.

The project uses Bob in two ways:

1. As an SDLC assistant during planning, design, implementation, testing, documentation, and demo preparation.
2. As the agentic interface for invoking the new Vault MCP Transit capabilities.

One-Sentence Pitch

We extended Vault MCP with Vault Transit support, giving Bob a new security capability for guiding and executing encryption-as-a-service workflows directly through Vault.

Problem Statement

Modern applications frequently need to encrypt sensitive data, but managing encryption keys directly introduces operational and security risk. Developers may need to handle key storage, rotation, access control, auditability, and cryptographic correctness. These requirements are easy to implement incorrectly.

HashiCorp Vault Transit solves this problem by providing encryption-as-a-service. Applications can send data to Vault for cryptographic operations while Vault protects and manages the encryption keys. However, interacting with Transit still requires users to understand Vault paths, payload formats, base64 encoding requirements, key versions, and specific API behavior.

The problem this project addresses is:

Developers and security teams need a simpler, guided, and agentic way to perform Vault Transit encryption workflows through Vault MCP without manually managing low-level Vault API details.

Solution

Our solution extends Vault MCP with a new set of Transit security tools. These tools allow Bob to assist users with Vault-backed encryption workflows through clear, structured operations.

The core workflow is:

```
Create Transit key
-> Read key metadata
-> Encrypt plaintext
-> Decrypt ciphertext
-> Rotate key
-> Rewrap old ciphertext with the latest key version
-> Generate and verify HMACs
```

This turns Bob into a practical Vault security assistant. Bob can guide the user, invoke the correct MCP tool, validate inputs, explain errors, and help document the workflow.

What We Built

We added a new Transit-focused capability area to Vault MCP. This is not a single encryption function; it is a coherent capability suite that supports key management, encryption operations, key lifecycle workflows, and data integrity verification.

Capability Area	Tool Examples	Purpose
Key management	create_transit_key, read_transit_key, rotate_transit_key	Create, inspect, and rotate Vault-managed Transit keys.
Encryption operations	encrypt_data, decrypt_data	Encrypt and decrypt data through Vault without exposing keys to the user or application.
Key rotation workflow	rotate_transit_key, rewrap_data	Demonstrate real key lifecycle management and update ciphertext to newer key versions.
Integrity checks	generate_hmac, verify_hmac	Generate and verify HMACs to confirm data integrity.

Core Transit Capabilities

Tool	Purpose	Vault API
create_transit_key	Create a new Vault Transit encryption key.	POST /{mount}/keys/{name}
read_transit_key	Read key configuration, type, and version metadata.	GET /{mount}/keys/{name}
encrypt_data	Encrypt base64-encoded plaintext using a Transit key.	POST /{mount}/encrypt/{name}
decrypt_data	Decrypt Vault ciphertext back into base64-encoded plaintext.	POST /{mount}/decrypt/{name}
rotate_transit_key	Rotate a Transit key to a new version.	POST /{mount}/keys/{name}/rotate
rewrap_data	Re-encrypt existing ciphertext using the latest key version.	POST /{mount}/rewrap/{name}
generate_hmac	Generate an HMAC for data integrity verification.	POST /{mount}/hmac/{name}
verify_hmac	Verify an HMAC against original input.	POST /{mount}/verify/{name}

Stretch Capabilities

Tool	Purpose
sign_data	Create a digital signature using a Transit signing key.
verify_signature	Verify a digital signature.
generate_random_bytes	Generate cryptographically secure random bytes.
hash_data	Generate a cryptographic hash.
list_transit_keys	List available Transit keys.

Why Vault Transit

- Security-relevant: it solves a real cryptographic key management problem.
- Demo-friendly: the workflow is easy to show end-to-end.
- Self-contained: it only requires Vault and does not depend on external cloud services.
- Technically meaningful: it includes key management, encryption, decryption, rotation, rewrapping, HMACs, and error handling.
- Reusable: the same pattern can be extended to other Vault engines later.

Implementation Structure

A clean implementation should add a dedicated Transit tool package and shared helpers for path construction, base64 validation, ciphertext validation, and Vault response handling.

```
pkg/tools/transit/  
├─ create_key.go  
├─ read_key.go  
├─ rotate_key.go  
├─ encrypt_data.go  
├─ decrypt_data.go  
├─ rewrap_data.go  
├─ generate_hmac.go  
├─ verify_hmac.go  
├─ transit_helpers.go  
└─ transit_test.go
```

Tool Specifications

Tool	Description	Parameters	Returns
create_transit_key	Create a new encryption key in the Transit engine.	mount, name, type, exportable, allow_plaintext_backup, derived	Key created successfully or Vault error.
read_transit_key	Read key configuration and metadata.	mount, key_name	Key type, versions, capabilities, and configuration.

Tool	Description	Parameters	Returns
encrypt_data	Encrypt plaintext data using a Transit key.	mount, key_name, plaintext, context, key_version, nonce	Vault ciphertext string such as vault:v1:...
decrypt_data	Decrypt ciphertext using a Transit key.	mount, key_name, ciphertext, context	Base64-encoded plaintext.
rotate_transit_key	Rotate an encryption key to a new version.	mount, key_name	New key version created; old ciphertext remains decryptable.
rewrap_data	Re-encrypt ciphertext with the latest key version.	mount, key_name, ciphertext, context	New ciphertext using the latest key version.
generate_hmac	Generate HMAC for data integrity verification.	mount, key_name, input, algorithm, key_version	HMAC signature string.
verify_hmac	Verify an HMAC signature.	mount, key_name, input, hmac, algorithm	Boolean validity result.

Bob’s Role Across the SDLC

This project intentionally uses Bob beyond basic code generation. Bob is used across the software development lifecycle to plan, implement, test, document, and demonstrate the new Vault MCP capability.

SDLC Phase	How Bob Was Used
Planning	Compared possible Vault MCP enhancements and helped select Transit as the best scoped security capability.
Design	Helped break the feature into core tools, stretch tools, inputs, outputs, and API mappings.
Implementation	Assisted with tool structure, naming, parameter validation, and Vault API request patterns.
Testing	Helped identify unit tests, integration tests, edge cases, and failure scenarios.
Documentation	Helped create user-facing docs, examples, troubleshooting notes, and demo scripts.
Demo preparation	Helped build a clear five-minute demo flow showing the value of the new capability.

Example Bob Prompts Used

Help us choose the best Vault MCP capability to implement for a Bob-a-thon project. Compare Transit, KV, PKI, and dynamic secrets based on usefulness, complexity, and demo quality.
Break down Vault Transit support into MCP tools with clear names, parameters, return values, and Vault API paths.
Help design a safe demo workflow for Vault Transit that shows create key, encrypt, decrypt, rotate, and rewrap.
Suggest unit and integration test cases for Vault Transit MCP tools, including base64 validation, invalid ciphertext, missing keys, and Vault API errors.

Help turn this implementation into a clear Bob-a-thon presentation that explains the problem, solution, Bob usage, and demo flow.

Agentic Best Practices

- 1. Tool-bounded actions: Bob does not directly manage secrets or keys. Bob invokes structured MCP tools that perform specific Vault operations.
- 2. Clear tool contracts: each Transit tool has defined parameters, expected return values, and known Vault API behavior.
- 3. Safe defaults: key creation should default to non-exportable keys and avoid plaintext key exposure.
- 4. Validation before execution: inputs such as mount paths, key names, ciphertext prefixes, and base64-encoded plaintext should be validated before Vault operations are attempted.
- 5. Human-readable feedback: Bob should return useful explanations when operations fail, such as missing Transit mounts, invalid key names, malformed ciphertext, or base64 encoding errors.
- 6. Reusable patterns: the Transit tool structure can serve as a template for adding future Vault MCP engines.

Testing Strategy

The testing strategy should prove that the Transit tools work independently and as a complete workflow.

Test Type	Coverage
Unit tests	Mock Vault HTTP responses, validate request payloads, validate parameter handling, and cover error paths.
Integration tests	Run against a real Vault dev server and test end-to-end create key, encrypt, decrypt, rotate, rewrap, generate HMAC, and verify HMAC flows.
Validation tests	Cover invalid base64, missing key names, empty mount paths, malformed ciphertext, and unsupported algorithms.
Regression tests	Ensure existing Vault MCP tools continue working and new Transit tools do not break current behavior.

Demo Plan

The demo should show Bob using the new Vault MCP Transit tools in a realistic encryption workflow. The key message is that Bob can now do more than explain Vault Transit; Bob can guide and execute Vault-backed security operations through MCP.

Step	Demo Action	Expected Result
1	Ask Bob to create a Transit key named customer-data.	Vault creates the key successfully.
2	Ask Bob to encrypt base64-encoded sample data.	Bob returns a Vault ciphertext string.
3	Ask Bob to decrypt the ciphertext.	Bob returns the original base64-encoded plaintext.

Step	Demo Action	Expected Result
4	Ask Bob to rotate the Transit key.	Vault creates a new key version.
5	Ask Bob to rewrap the old ciphertext.	Bob returns new ciphertext using the latest key version.
6	Optionally ask Bob to generate and verify an HMAC.	Bob confirms the data integrity check is valid.

Five-Minute Video Structure

Time	Segment	What to Show
0:00-0:30	Problem	Apps need encryption, but key management is risky and hard to implement correctly.
0:30-1:15	Solution	Vault Transit solves encryption-as-a-service; the project adds Transit support to Vault MCP.
1:15-2:15	Implementation	Show the Transit tool package and explain the core tools.
2:15-4:00	Live workflow	Create key, encrypt, decrypt, rotate, and rewrap.
4:00-4:40	Testing and safety	Show tests and mention validation, safe defaults, and error handling.
4:40-5:00	Close	Bob now has a practical Vault security capability through MCP.

Implementation Timeline

Phase	Work Items	Estimated Time
Phase 1: Core encryption	Set up transit package, implement create/read key and encrypt/decrypt.	8-10 hours
Phase 2: Lifecycle workflow	Implement rotate key, rewrap data, generate HMAC, and verify HMAC.	6-8 hours
Phase 3: Tests	Add unit tests, integration tests, and validation/error cases.	4-6 hours
Phase 4: Polish and demo	Improve docs, examples, troubleshooting, and final video script.	4-6 hours

Risk Assessment

Risk	Probability	Impact	Mitigation
Base64 encoding errors	Medium	Low	Validate input and provide clear error messages.
Ciphertext format issues	Low	Low	Validate vault:v prefix before decrypt and rewrap operations.
Key version confusion	Low	Medium	Expose key metadata and clearly show rotation and rewrap behavior in the demo.
Scope creep	Medium	Medium	Prioritize the eight core Transit tools before stretch capabilities.

Risk	Probability	Impact	Mitigation
Insufficient Bob framing	Medium	High	Show both how Bob helped build the feature and how Bob can now use it.

Success Metrics

Category	Metric
Implementation	All eight core Transit tools implemented and callable through Vault MCP.
Workflow	End-to-end create key, encrypt, decrypt, rotate, rewrap, generate HMAC, and verify HMAC flow works.
Testing	Unit and integration tests cover successful flows and failure paths.
Documentation	README, tool docs, usage examples, troubleshooting, and demo script are included.
Bob usage	Bob usage across planning, design, coding, testing, documentation, and demo preparation is documented.
Demo	Five-minute video clearly shows the problem, solution, implementation, and working workflow.

Recommended Repo Documentation

The repository should include documentation that supports both the technical implementation and the contest rubric.

```
README.md
docs/problem-and-solution.md
docs/bob-usage-log.md
docs/agentic-best-practices.md
docs/demo-script.md
docs/transit-tool-examples.md
docs/testing-strategy.md
```

Final Framing

The project should not be presented as a small encryption utility. The strongest framing is that the team extended Vault MCP with a complete Vault Transit capability suite and used Bob as both the development assistant and the agentic security interface.

Recommended wording:

Our project extends Vault MCP with a complete Vault Transit capability suite. With Bob as the agentic interface, users can create and inspect encryption keys, encrypt and decrypt data, rotate keys, rewrap ciphertext to newer key versions, and verify data integrity using HMACs. This turns Bob from a passive assistant into an active security workflow interface for Vault-backed encryption-as-a-service.

Conclusion

This project is a strong Bob-a-thon submission because it shows both sides of agentic development: Bob helped build the capability, and Bob can now use the capability. The result is not just a code addition. It is a reusable security workflow that gives Bob a new practical function inside the Vault ecosystem.